TRAINING CHESS BOTS WITH VARIOUS MACHINE LEARNING TECHNIQUES

by

BHAVYA SINGH
URN: 6839571

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Dr Joey Sik Chun Lam

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Bhavya Singh
May 2026

Abstract

Write a summary of the work presented in your dissertation. Introduce the topic and highlight your main contributions and results. The abstract should be comprehensible on its own, and should no contain any references. As far as possible, limit the use of jargon and abbreviations, to make the abstract readable by non-specialists in your area. Do not exceed 300 words.

Acknowledgements

Write any personal words of thanks here. Typically, this space is used to thank your supervisor for their guidance, as well as anyone else who has supported the completion of this dissertation, for example by discussing results and their interpretation or reviewing write ups. It is also usual to acknowledge any financial support received in relation to this work.

Contents

List of Figures

List of Tables

Glossary

A_f	The source message, being a sequence of f source symbols $a_1 a_2 \dots a_f$
a_i	The i^{th} symbol in the source message, where $a_i \in S_m$
B_g	The decoded message, being a sequence of g source symbols $b_1 b_2 \dots b_g$
b_i	The i^{th} symbol in the decoded message, where $b_i \in S_m$
C_h	The transmitted (compressed) message, being a sequence of h Tunstall codewords $c_1 c_2 \dots c_h$
c_i	The i^{th} codeword in the transmitted message, where $c_i \in T_n$
D_h	The received (compressed) message, which for a complete Tunstall code is a sequence of h Tunstall codewords $d_1 d_2 \dots d_h$
d_i	The i^{th} codeword in the received message, where $d_i \in T_n$ for a complete Tunstall code

Abbreviations

BER	Bit Error Rate
BPSK	Binary Phase Shift Keying
BSC	Binary Symmetric Channel
DCT	Discrete Cosine Transform
ECC	Error Correcting Codes
FEC	Forward Error Correction
JPEG	Joint Photographic Experts Group
MPEG	Moving Pictures Experts Group
SER	Symbol Error Rate
SNR	Signal to Noise Ratio

Chapter 1

Introduction

1.1 Problem Statement

For decades, the game of chess has been a benchmark of Artificial Intelligence progress — a controlled environment used to measure the evolution of machine logic. Early milestones, such as IBM's Deep Blue, relied on carefully handcrafted heuristics and "brute force" calculations. While effective, these systems were inherently restrictive, struggling to generalize beyond the specific rules encoded by their creators.

Today, the landscape is dominated by neural-heavy architectures like AlphaZero, MuZero, and Leela Chess Zero. These engines have pushed ELO ratings beyond the 3500 mark, effectively concluding the era of human-computer competition. However, this superiority comes at a cost: modern engines are increasingly reliant on massive computational overhead and deep search trees to maintain their edge.

1.2 Motivation

The current state-of-the-art engines are outstanding, but they are also "computationally greedy." Their strength is acquired from exploring millions of potential future states (search) before making a move. This reliance on search constitutes a difficult obstacle to entry for mobile or edge-computing applications.

The motivation lies in the pursuit of "Search-less Chess." By leveraging Convolutional Neural Networks (CNNs), Transformers, and Graph Neural Networks (GNNs). We aim to investigate whether a model can develop a "grandmaster's intuition"—the ability to look at a board and immediately identify the optimal move without the safety net of a search algorithm. This would not only drastically reduce move latency but also offer insights into how deep learning architectures internalize complex spatial and strategic relationships.

1.3 The Challenge of Chess

Chess is a game of perfect information, yet it remains computationally "unsolved" because of its high strategic depth. A player must simultaneously balance tactical immediate threats (checks, captures) with abstract long-term goals (piece mobility, pawn structure, and king safety).

1.3.1 Computational Complexity

The complexity of chess is often illustrated by Shannon's Number, which estimates the game tree complexity at approximately 10^{120}. Even the state space—the number of possible legal positions—is estimated at 10^{40}. Because the number of possible variations grows exponentially with every half-move (ply), brute-force calculation is a physical impossibility. This necessitates an approach that can prune the search space or, as we propose, bypass it entirely through high-fidelity policy prediction.

1.3.2 Failure of Naive Approaches

Simple heuristics (e.g., assigning a point value to pieces) fail because chess is highly contextual; a Queen is worthless if the King is in an inescapable checkmate. Naive machine learning models often fall into the trap of "greedy" play—capturing material while ignoring a looming positional collapse. Without the "look-ahead" provided by search, a model must be significantly more sophisticated to recognize the subtle cues that distinguish a winning position from a losing one.

1.4 Critique of Existing Solutions

Current engines like Stockfish 18 are incredibly powerful but function as "black boxes" of search optimization. They require significant CPU/GPU resources to reach their peak ELO. Furthermore, even hybrid models like Leela Chess Zero—which use neural networks for evaluation—still rely on Monte Carlo Tree Search (MCTS) to validate their "hunches."

1.5 Overview of My Approach

1.5.1 Machine Learning Architectures

This project explores an architectural comparison to determine which structure best captures the tactical and strategic nuances of chess without search.

- **CNNs:** To treat the board as a 2D image, capturing local spatial patterns and piece interactions.
- **Transformers:** To leverage self-attention mechanisms, allowing the model to understand long-range dependencies across the board (e.g., a Bishop's influence from a8 to h1).
- **GNNs:** To model the board as a dynamic graph where pieces are nodes and their legal moves are edges, emphasizing the relational geometry of the game.

We will train these models using the Lichess Stockfish dataset, effectively "distilling" the knowledge of a 3500-ELO engine into our search-less architectures.

1.5.2 Comparison Methodology

To validate our approach, we will benchmark our models against Stockfish at varying "depth" constraints. This allows us to simulate different ELO ranges and see exactly where a search-less model begins to falter compared to a searching engine. Additionally, we will use the Lichess Puzzle dataset to test the models on specific tactical motifs, providing a granular look at their "chess IQ" in areas like forks, pins, and endgame transitions.

1.6 Summary of Key Results

1.7 Scope and Limitations

The goal of this research is not to compete with the world's strongest engines, but to push the ceiling of what is possible within the "search-less" paradigm.

Limitations: We acknowledge that without search, our models may struggle with "horizon effects," where a tactic requires a deep calculation beyond the model's immediate pattern recognition. Similarly, highly technical endgames (e.g., Rook, and Bishop vs. Rook) may remain a challenge for a purely intuitive model.

1.8 Aims and Objectives

The primary objective is to engineer and evaluate a suite of deep learning models (CNN, Transformer, GNN) capable of achieving a 2000 ELO rating without the use of search algorithms. Success will be measured by:

1. **Inference Speed:** Demonstrating a significant reduction in move generation time compared to traditional engines.

2. **Generalization:** Proving the model has "learned" chess principles rather than memorized positions, verified via performance on unseen puzzle sets.

3. **Comparative Analysis:** Identifying which architecture (spatial, relational, or sequential) is best suited for the unique geometry of chess.

1.9 Report Structure

The remainder of this report is organized as follows: **Chapter 2** surveys the history of chess AI and the technical evolution of deep learning in games. **Chapter 3** details our methodology, from data curation and normalization to the specific hyperparameters of our three architectures. **Chapter 4** presents a rigorous analysis of the results, including ELO benchmarking and tactical performance. Finally, **Chapter 5** discusses the implications of these findings for the future of lightweight, intuitive AI.

Chapter 2

Literature Review

2.1 Classical Chess Engines

Earlier Chess engines relied heavily on the minimax algorithm combined with alpha-beta pruning to efficiently search the game tree.

These engines utilized handcrafted evaluation functions that assessed board positions based on material count, piece activity, king safety, and other heuristics Example Deep Blue.

While effective, these approaches had limitations in terms of adaptability and often required extensive domain knowledge to fine-tune the evaluation parameters. Stockfish, one of the most powerful open-source chess engines, exemplifies this classical approach with its highly optimized search algorithms and evaluation function.

2.2 Probabilistic Search Methods

Monte Carlo Tree Search (MCTS) is a probabilistic search method that has been successfully applied to various games, including chess. MCTS builds a search tree based on random sampling of the game space, allowing it to explore promising moves without exhaustive search. This approach has advantages over traditional minimax algorithms, particularly in complex game states where the search space is vast. MCTS has been used in engines like Leela Chess Zero, which combines MCTS with neural network evaluations to achieve strong performance.

Additionally, MCTS can adapt its search strategy based on the outcomes of previous simulations, making it more flexible in dynamic game environments. This adaptability has contributed to its success in various game AI applications beyond chess, such as Go and general game playing.

2.3 The Deep Learning Revolution

AlphaZero, developed by DeepMind, marked a significant breakthrough in chess AI by combining deep learning with self-play reinforcement learning. AlphaZero's architecture consists of a policy network that predicts the probability of winning moves and a value network that estimates the expected outcome of a position. Through self-play, AlphaZero was able to learn and improve its performance without relying on human game data, achieving superhuman performance in chess, shogi, and Go.

This approach has inspired the development of open-source implementations like Leela Chess Zero, which utilizes a similar architecture and training methodology to achieve competitive performance against traditional engines. The success of deep learning in chess has also spurred research into more advanced neural network architectures and training techniques, further pushing the boundaries of what AI can achieve in strategic games.

2.4 Supervised Learning Approaches

DeepChess uses a two-stage deep neural network: a Pos2Vec autoencoder pre-trained unsupervised on 640K human games from the CCRL database, followed by a supervised comparison network that takes two positions as input and outputs which is more favourable. The system never assigns numerical evaluations — it directly learns to rank positions, reaching grandmaster-level play with no hand-crafted heuristics.

Giraffe trains a neural network evaluation function using temporal-difference learning via self-play, with minimal hand-crafted input features. Despite receiving no explicit human chess knowledge, the resulting engine plays at approximately FIDE International Master level $top 2.2\% of rated players$, comparable to the strongest handcrafted engines of the time. Key distinctions from DeepChess: Giraffe uses self-play (not human game databases) and TD-learning rather than supervised pairwise classification.

2.5 Recent Innovations

Graph representations of chess positions have been explored to capture the relational structure of the game more effectively. Tomas Rigaux and Hisashi Kashima (2024) proposed a graph-based neural network architecture that models the interactions between pieces and their positions on the board, leading to improved evaluation accuracy and strategic understanding.

Explainable AI (XAI) has also gained attention in chess, with researchers like Svendsen and Tørresen (2022) developing methods to provide insights into the decision-making processes of chess engines. This includes techniques for visualizing the influence of different pieces and moves on the engine's evaluations, helping players

and researchers understand the underlying strategies employed by AI agents.

2.6 Game AI Frameworks

The library provides move generation and validation, PGN parsing and writing, Polyglot opening book reading, Syzygy/Gaviota tablebase probing, and full UCI/XBoard engine communication - essentially everything needed to interface with any standard chess engine in Python.

2.7 Web-Based Chess Platforms

2.7.1 Existing Interactive Chess Platforms

Thibault Duplessis (lead dev), Lichess / Lila — open-source chess server. Lichess is written in Scala/Play with a fully asynchronous architecture using Akka actors, MongoDB for game storage (800M+ games), Elasticsearch indexing, and nginx proxying both HTTP and WebSocket connections. The real-time move streaming is handled natively via WebSockets.

2.7.2 Bot-vs-Bot Comparison Platforms

Chessprogramming Wiki — "Tournament Manager" entries and TCEC (Top Chess Engine Championship) as a descriptive reference for bot-vs-bot spectator design. Search "TCEC chess engine competition" for the definitive real-world example of a spectator-oriented bot-arena platform.

2.7.3 Human-vs-Bot Interfaces

Lichess API Board Endpoints — boardGameStream / boardGameMove provide the reference implementation for streaming live moves from a bot to a human player in real time. The API is REST+NDJSON.

2.7.4 Web Technologies for Real-Time Games

"The WebSocket Protocol." IETF RFC 6455, 2011. The normative standards reference — cite when defining the protocol your game communication layer uses. Available at tools.ietf.org/html/rfc6455.

Using Go and Svelete for the frontend and backend respectively is a novel choice that offers performance benefits and a modern development experience. Go's concurrency model is well-suited for handling multiple simultaneous games and real-time interactions, while Svelte's reactive framework allows for efficient UI updates with

minimal overhead. This combination is less common in game development compared to traditional JavaScript frameworks.

2.8 Critical Analysis

Exploring the Latest Neural Network Architectural Developments in Chess AI," 2024 — confirms CNNs remain competitive in hybrid $NNUE + \alpha - \beta$ deployments and are the most computationally efficient at inference time.

CNNs have a limited receptive field: a 3x3 kernel only sees adjacent squares, requiring many stacked layers to propagate information across the board. This is structurally mismatched to chess, where a single piece *e.g.*, $arookorqueen$ influences the entire board instantly.

Primary citation for the Square Transformer paradigm. Chessformer (CF-240M) achieves 2347 Elo and 93.5% puzzle accuracy at 30x fewer FLOPs than comparable AlphaZero-scale models. The critical finding is that positional encoding is the decisive design choice — absolute, relative, and dynamic $Shawetal.$ encodings produce markedly different results.

Trains a large transformer on 10 million Lichess games annotated with Stockfish 16 at 50ms per position — architecturally the closest existing work to your own pipeline. Discusses action-value and state-value heads and the tradeoffs between square-based and piece-based tokenisation strategies. Critical gap your work addresses: Ruoss et al. use a single architecture; you systematically compare Square and Piece Transformer under identical conditions.

Quadratic attention complexity over 64 tokens is manageable but costly at scale compared to CNN inference.

Transformers require careful positional encoding — absolute embeddings fail to capture board geometry and naive relative encodings lose piece-type information.

AlphaGateau (GAT + GATEAU edge-feature layer) outperforms ResNet-based AlphaZero by an order of magnitude in sample efficiency on a 5x5 chess variant, demonstrating that graph representations encode relational structure (legal moves as edges) that grids cannot express compactly.

Rigaux & Kashima train only on a 5x5 variant and fine-tune to 8x8 — no published paper presents a full GCN/GAT chess engine trained end-to-end on standard chess from scratch against a CNN and Transformer baseline on the same dataset. This is one of the most direct gaps your project fills.

2.9 Summary

Representation matters more than depth — Chessformer shows position encoding is the decisive variable for transformers; Rigaux & Kashima show graph edges (legal moves) encode relational structure that grids miss. Your multi-architecture comparison is the natural empirical test of this claim.

Supervised Stockfish annotation is a viable and scalable training signal — Ruoss et al., Molinelli et al., and Oshri & Khandwala all validate this pipeline, but none investigate PV unrolling for position diversity.

No fair cross-architecture benchmark exists — every existing paper proposes one architecture in isolation. Your project's controlled comparison on identical Lichess/Stockfish data with PV-unrolled positions is the gap that makes it publishable.

Chapter 3

Methodology and Design

3.1 Requirements Analysis

3.2 Framework Architecture

3.2.1 AI Agent Module

3.2.2 Training and Evaluation Engine

3.3 Design Decisions and Justification

3.4 Chess Implementation

3.5 Web Platform Design

3.5.1 Platform Requirements

3.5.2 System Architecture

3.5.3 Game Modes

3.5.4 User Interface Design

3.5.5 Backend API Design

3.6 Technical Challenges

3.7 Diagrams

Chapter 4

Implementation

4.1 Development Environment and Tools

4.2 Core Framework Implementation

4.3 Supervised Learning Model

4.3.1 Data Preparation

4.3.2 Neural Network Architecture

4.3.3 Training Process

4.4 Reinforcement Learning Model

4.4.1 AlphaZero-Style Architecture

4.4.2 Training Loop

4.5 Web Platform Implementation

4.5.1 Frontend Development

4.5.2 Backend Development

4.5.3 Bot-vs-Bot Mode

4.5.4 Human-vs-Bot Mode

4.5.5 Model Selection and Configuration

Chapter 5

Evaluation

5.1 Evaluation Methodology

5.2 Supervised Learning Model Results

5.2.1 Training Performance

5.2.2 Playing Strength

5.2.3 Error Analysis

5.3 Comparative Analysis

5.4 Ablation Studies

5.5 Web Platform Evaluation

5.5.1 Usability Testing

5.5.2 Performance and Responsiveness

5.5.3 Bot-vs-Bot Match Results via Platform

5.6 Comparison with Related Work

5.7 Interpretation and Significance

5.8 Limitations and Challenges

Chapter 6

Critical Review and Reflection

6.1 Achievement of Objectives

6.2 What Worked Well

6.3 What Didn't Work Well

6.4 Lessons Learned

6.5 Personal Reflection

6.6 Future Work

6.6.1 Short-term Extensions

6.6.2 Web Platform Enhancements

6.6.3 Long-term Research Directions

6.7 Broader Impact

Chapter 7

Legal, Social, Ethical, and Professional Issues

7.1 Ethical Considerations

7.1.1 Research Ethics

7.1.2 Responsible AI

7.2 Legal Issues

7.2.1 Intellectual Property

7.2.2 Copyright and Licensing

7.3 Social Issues

7.3.1 Accessibility

7.3.2 Web Platform Accessibility

7.3.3 Impact on Chess Community

7.4 Professional Issues

7.4.1 Code of Conduct

7.4.2 Information Security

7.4.3 Web Platform Security

7.5 Sustainability

Chapter 8

Conclusion

Appendix A

Code Listings

Appendix B

Additional Results

Appendix C

User Guide

Appendix D

Game Examples

Appendix E

Ethical Approval Documentation

Appendix F

Project Planning Documents

Bibliography

Briffa, J. A., Schaathun, H. G. & Wesemeyer, S. (2010), An improved decoding algorithm for the Davey-MacKay construction, *in* 'Proc. IEEE Intern. Conf. on Commun.', Cape Town, South Africa.

Burrows, M. & Wheeler, D. J. (1994), A block-sorting lossless data compression algorithm, Technical report, Digital SRC Research Report.

el Gamal, A. A., Hemachandra, L. A., Shperling, I. & Wei, V. K. (1987), 'Using simulated annealing to design good codes', *IEEE Transactions on Information Theory* **33**(1), 116–123.

Farrell, P. G. (1992), 'Notes on source coding'. University of Manchester.

Henderson, B. (2009), *Netpbm*.
URL: `http://netpbm.sourceforge.net/doc/`

NVI (2009), *NVIDIA CUDA Programming Guide*. Version 2.3.

Press, W. H., Teukolsky, S. A., Vetterling, W. T. & Flannery, B. P. (1992), *Numerical Recipes in C: The Art of Scientific Computing*, second edn, Cambridge Universty Press.

Tunstall, B. P. (1968), Synthesis of Noiseless Compression Codes, PhD thesis, Georgia Institute of Technology.